

TextQuant

Turning corporate environmental disclosure into a cross-sectional, point-in-time signal

HackMIT 2026 — solo submission

September 2026

Alternative data improves situational awareness and decision-making, but it also generates noise. TextQuant turns corporate environmental disclosure into a cross-sectional, point-in-time signal. It reads any sustainability report, splits it into atomic claims, and classifies each one by **which piece of verifying information is missing** — using three parallel NLP tracks (rule-based, statistical, deep learning) plus anomaly detection over the document's own baseline.

The target is not false statements. Sustainability reports rarely contain those. They state a real figure and omit the one thing that would let you check it: the accounting method, the scope boundary, the absolute total, the definition of the term.

Repository: <https://github.com/PeacoFoxy/hackmit-greenwash-audit>

What it found

Four numbers, one formula each, reproducible offline with no API key.

	Formula	Result
Document that needs no analyst attention	$1 - \text{claims surfaced} / \text{sentences}$	97.8%
Quantified commitments that cannot be progress-checked	$\text{targets with } <2 \text{ observations} / \text{targets}$	96.6%
Rule tree and language model reaching the same label	$\text{CONFIRMED} / (\text{CONFIRMED} + \text{CONTESTED})$	78.6%
Quantified claims technically true but missing their basis	$C / (A + C)$, precision-corrected	45–49%

The last one is an interval on purpose. $C / (A + C)$ comes to 80.9%, but class-C precision is 0.667 on validated rules and 0.250 on the ten the blind test exposed, so quoting the raw count would carry that error into the headline. Each C is weighted by the measured precision of the rule behind it. It is the only one of the four that depends on the classifier being right — the other three are counting facts.

Inspiration

Corporate text is growing exponentially and is expensive to read precisely — 50 to 120 pages per sustainability report, hundreds of reports per portfolio. Doing it by hand does not scale, and keyword scanning does not work, because the problematic sentences contain no suspicious keywords. They are true. I wanted an algorithm that finds the *absence* of information rather than the presence of bad words.

What it does

Any environmental report (PDF)

1. PREPROCESSING

text extraction, sentence split, prose filter

2. FEATURE EXTRACTION

7 language signals + 9 accounting rules per sentence

3. ANOMALY DETECTION

flag density vs the report's own baseline -> passages

4. CLAIM EXTRACTION

atomic, independently checkable assertions

5. THREE PARALLEL TRACKS

Track R

rule decision tree, 22 nodes / 26 terminals

0 model calls

Track S

term x qualifier lift statistics

0 model calls

Track D

language model, four-class rubric

CONSENSUS LAYER – agreement between tracks

as a label-free confidence signal

6. SIGNAL OUTPUT

labels · passages · indicators · recommended action

Every claim gets one of four labels, graded like a credit rating rather than judged true/false:

Label	Meaning
A	Substantiated — concrete figure with its scope or method stated
B	Vague — no verifiable quantity
C	Technically true but incomplete — the figure is real, the basis is missing
D	Contradicted by other disclosed information

Class C is the whole point. Without the rubric, a language model finds **zero** class-C claims — precision, recall and F1 all exactly 0.000 — while its overall accuracy still looks respectable, because always answering B scores well.

Beyond the NLP pipeline, TextQuant is built around four ways text defeats a reader, each absorbed by a different track:

Challenge	Which track absorbs it
Biased training data	Track R is not trained at all — its rules encode GHG Protocol concepts, so they cannot inherit a corpus's habits
Misinterpretation	Track R ships the root-to-leaf path with every label, so a wrong reading is visible rather than silent
New vocabulary	Track S learns term × qualifier lift from the document's own corpus, so an unseen term still gets scored
Tone of voice	The rubric grades what is disclosed, never how assertively. A confident sentence with no boundary is still C

How I built it

I started from the AI hallucination problem: how do you verify that a model's output is the one you wanted? I looked at how Moody's rates credit and applied the same idea — a graded scale measuring **alignment** rather than correctness, which is what makes a disagreement between tracks informative instead of just wrong.

That design choice is why the rule track carries the explanation. Every label ships with the path that produced it, so a judgment can be audited instead of trusted.

Validation corpus: 3 reports, 4,084 sentences, 390 extracted claims, 49 blind-annotated labels.

	Rule tree	LLM, no rubric	LLM, with rubric
Accuracy	0.793	0.586	0.690
Class-C F1	0.800	0.000	0.667
Model calls per 100 claims	0	20.7	20.7
Cost per 100 claims	\$0.00	\$0.037	\$0.042

Individual contributions

The whole project: the idea, requirements analysis, design, implementation, data backtesting, algorithm optimization, and submission documents. All three teammates left for other teams three days before the hackathon started.

Challenges I ran into

Calibration — how do I know the classification aligns with what an analyst actually needs? At first I assumed the only answer was human supervision. Then, while optimizing, I read a paper on cross-checking model output (Hicks et al., PMC11404377) and implemented it: if the disclosure a rule says is missing appears in a nearby sentence, retract the flag.

I measured it, and it made things worse. Class-C balanced accuracy fell $0.868 \rightarrow 0.697$. Of 8 claims it cleared, 1 was right. The cause turned out to be interesting: phrases like *global operations* recur every few sentences in these reports, so “appears nearby” never proved the qualifier was bound to *that* figure — which is exactly the misleading pattern class C exists to catch. The implementation is kept in the repository with its measurements, switched off.

That failure changed my approach. I built a statistical backtesting harness so every subsequent idea had to survive measurement before shipping. **Thirteen optimization rounds: 6 shipped, 7 rejected on their own numbers.**

Round	Result	Outcome
1 · Drop a near-uniform flag	Detection threshold fell 27–42%; passages found rose $11 \rightarrow 13$	Shipped
2 · Add specificity / balanced accuracy / F-beta	Exposed that class-C recall of 1.000 was bought with specificity of 0.550	Shipped
3 · Accept named facility boundaries, not only scope terms	Accuracy 0.690 $\rightarrow$ 0.793	Shipped
6 · Feed rule flags to the model as hints	20% more input tokens, 3.5 points <i>lower accuracy</i>	Rejected
7 · Context cross-check	$0.868 \rightarrow 0.697$	Rejected
8 · Ensemble voting	16 combinations, none beat the rule tree alone	Rejected
9 · Abstention	5 confidence proxies, none beats random abstention	Rejected
11 · TF-IDF + linear SVM	0.483 under leave-one-out vs a 0.448 majority baseline	Rejected
12 · Blind-test rules the gold set never exercised	Validated rules: 0.667 precision. Unvalidated: 0.250 ($p = 0.024$)	Shipped as a measurement

The second hard problem was **not fooling myself**. Round 13 is the example I’d point to: a confidence signal that beat its control at 0.806 vs 0.710 — and turned out to be an artifact of how the samples were drawn. A source-stratified control removed the effect entirely. I recorded it as untestable rather than as a win, and built the confound check into the code so nobody misreads it later.

Accomplishments I'm proud of

- An algorithm that finds what is *missing* from a claim, with an auditable path behind every judgment — **at zero marginal model cost**, beating a language model on the same task.
 - **Reproducibility with no API key.** All 175 model responses are cached and committed, so anyone can clone the repository and reproduce every reported number offline. I verified this from a fresh clone in a clean virtual environment.
 - **Rejecting seven of my own ideas on the numbers**, including the one the reference paper suggested. First time focusing purely on algorithm development and running an entire project end to end.
 - One result that needs no accuracy claim at all: **57 of 59 quantified commitments in these reports cannot be progress-checked from the report that states them.**
-

What I learned

Statistical honesty is a skill, not an attitude. I twice nearly shipped a false positive. Both times the fix was a better control, not a better intention. Now every result in the repository states what it does *not* establish — including that the five methods tested are statistically indistinguishable here (Friedman $\chi^2 = 4.82$, $p = 0.31$). The rule tree scores highest; I do not claim it is superior.

“Not measured” is not “probably fine.” Ten of my rules had never met a human label. I built a stratified blind test for exactly those, and they scored 0.250 precision against 0.667 for the rest. Assuming they were fine would have been wrong.

Focus. You cannot make the interface, the backend and the algorithm all excellent under a deadline. I spent the end of day one on interface design, realized the time budget didn't allow it, and went back to the algorithm. That was the right trade.

Never give up. All three teammates left three days before the hackathon. I still finished most of the project alone. You don't know your upper limit until you're forced to find it.

What's next

1. **External evidence.** Today every claim is checked against the document that makes it — which is precisely why the 57-of-59 result exists. Next is retrieval (BM25 + vector) against SEC filings, CDP responses and certification registries, so a claim can be verified against a *different* source rather than only inspected.
2. **Fix the rules the blind test exposed.** Ten rules scored 0.250. They need more annotation before they can be repaired or removed — 1–2 labels each is enough to stop calling them validated, not enough to act on.

3. **Time series.** One reporting year means no comparison against prior disclosures. The strongest greenwashing signal is a boundary that quietly changes between years — invisible in a single report, obvious across three.
4. **Scale the validation.** 49 blind labels is the binding constraint on every conclusion here. The method is general; the measured accuracy is not yet a population estimate.
5. **Portfolio-level output.** The indicators are already comparable across reports. The next step is ranking a full universe and testing whether the signal has any relationship to realized outcomes.